```
(requires: b>=0 )
(returns: a to the power of b)
let rec pow a b:=
        if b=0 then 1
        else a * pow a (b-1)
```

In the above function definition variable *'b'* is the parameter and the value which is passed to the variable *'b'* is the argument. The precondition ***(requires)*** and postcondition ***(returns)*** of the function is given. Note we have not mentioned any types: ***(data types)***. Some language compiler solves this type ***(data type)*** inference problem algorithmically, but some require the type to be mentioned.

In the above function definition if expression can return ***1*** in the then branch, shows that as per the ***typing*** rule the entire if expression has type ***int***. Since the if expression is of type ***'int'***, the function's return type also be *'int'*. ***'b'*** is compared to *0* with the equality operator, so ***'b'*** is also a type of *'int'*. Since ***'a'*** is multiplied with another expression using the * operator, ***'a'*** must be an int.

2. Parameter with Type

Now let us write the same function definition with types for some reason:

```
(requires: b>=0 )
(returns: a to the power of b )
let rec pow (a: int) (b: int) : int :=
        if b=0 then 1
        else a * pow a (b-1)
```

When we write the type annotations for ***'a'*** and ***'b'*** the parentheses are mandatory. Generally we can leave out these annotations, because it's simpler to let the compiler infer them. There are times we may want to explicitly write down types. This is useful on times when you get a type error from the compiler that doesn't make sense. Explicitly annotating the types can help with debugging such an error message.

The syntax to define functions is close to the mathematical usage: the definition is introduced by the keyword ***let***, followed by the ***name*** of the function and its *arguments*; then the formula that computes the image of the argument is written after an := sign. If you want to define a recursive function: use ***"let rec" instead of "let".***

Syntax: The syntax for function definitions:

```
let rec fn a1 a2 ... an := k
```

Here the ***'fn'*** is used as a function name. The names ***'a1'*** to ***'an'*** are variables used as parameters. The keyword ***'rec'*** is required if ***'fn'*** is to be a recursive function; otherwise it may be omitted.

Note

A function definition which call itself is called recursive function.

For example: let us see an example to find the factorial of a number.

12th Computer Science_EM Chapter 1.indd 2 23-12-2022 15:42:16